

Fundamentals of API Development

Capstone: Evolve and Protect a Card Applications API

Day 4 - 60-minute breakout

Capstone Goal

Build one credit union API through the full course lifecycle:

1. Design three visible contracts and classify their audiences
2. Keep `v1` available while announcing its retirement
3. Document the API through OpenAPI
4. Apply authentication, authorization, and rate limiting
5. Prove the behavior with a CI-ready JUnit5 suite

Your Working Project

Open `quickstart-project/breakouts/capstone/` in `api-dev-setup`.

Path	Purpose
<code>README.md</code>	Timing, goals, and definition of done
<code>01</code> through <code>04</code> guides	Decisions, code, security, and test prompts
<code>start/</code>	Compiling project with seven TODOs and acceptance tests
<code>solution/</code>	Complete reference implementation

Use synthetic data only. This classroom policy is not a real lending model.

Start with a Failing Gate

```
cd api-dev-setup/quickstart-project/breakouts/capstone/start  
mvn -q -DskipTests compile  
mvn test
```

The starter compiles, but the acceptance suite fails at incomplete behavior.

KEEP THE CONTRACT

Do not delete, skip, or weaken a test. Correct production code against the executable contract.

60-Minute Run of Show

Time	Activity
0-5	Compile and inspect failing tests
5-12	Decide audience and breaking changes
12-30	Complete <code>v1</code> , <code>v2</code> , and <code>v3</code> behavior
30-42	Configure Spring Security role rules
42-50	Implement the rate-limit TODO
50-60	Run tests, inspect OpenAPI, prepare share-out

Basic Goal

Design the evolution, implement the contracts, retire **v1** safely

One Resource, Three Contracts

Version	Audience	Response purpose
v1	Member	Legacy <code>eligible</code> result
v2	Member	Decision ID, outcome, limit, and reasons
v3	Underwriter	Ratio, risk tier, and detailed decision

Path: `POST /api/v{n}/card-applications/decisions`

Ask whether every difference truly needs a new version. A richer optional field alone can be non-breaking.

Prompt 1: Review Before Coding

Open `01-design-the-evolution.md` and copy its prompt.

Ask the LLM to:

1. Compare the three contracts without editing files
2. Classify each transition as breaking or non-breaking
3. Identify sensitive fields that should not be logged
4. Challenge the assumption that every new capability needs a version

HUMAN CHECKPOINT

Name the exact consumer behavior that breaks. "It is a new feature" is not enough.

Decision Rules

- Decline below score `620` or above debt ratio `0.45`
- Review below score `680` or above debt ratio `0.36`
- Approve all other valid applications
- Cap approved limit at `20%` of annual income
- Calculate with `BigDecimal` and deterministic rounding
- Never log or echo unnecessary applicant data

The rules are intentionally simple and synthetic so the lab tests API engineering, not underwriting.

Prompt 2: Complete TODOs 1-5

Open `02-build-the-versions.md` and use its bounded prompt.

Require the LLM to preserve:

- Package names, public signatures, model records, and paths
- Existing validation and all acceptance tests
- Exact decision rules and `BigDecimal` calculations
- No dependencies, persistence, external calls, or request logging

Run the three focused version tests before moving on.

Sunset Without a Surprise

Keep `v1` functional and return:

```
Deprecation: @1793491200  
Sunset: Thu, 31 Dec 2026 23:59:59 GMT  
Link: </api/v2/card-applications/decisions>; rel="successor-version"
```

A retirement plan has three stages: soft deprecation, hard deprecation, then sunset.

Removing `v1` today is not a lifecycle strategy.

Intermediate Goal

Enforce who you are, what you may do, and how often

Spring Security Access Matrix

Path	MEMBER	UNDERWRITER	Anonymous
/api/v1/**	200	200	401
/api/v2/**	200	200	401
/api/v3/**	403	200	401

Training users and HTTP Basic are preconfigured scaffolding.

A production financial API should use stronger identity, short-lived tokens, and object-level authorization.

Prompt 3: Complete TODO 6

Open `03-secure-and-limit.md` and use the authorization prompt.

Ask the LLM to:

1. Configure ordered matchers for the access matrix
2. Keep OpenAPI and Swagger UI public for this lab
3. Preserve password encoding, HTTP Basic, and filter order
4. Explain why anonymous is `401` and wrong role is `403`
5. Run only the two focused security tests first

Rate Limiting Comes Third

The request order is fixed:

1. Authentication: reject unknown identity with `401`
2. Authorization: reject insufficient role with `403`
3. Rate limit: reject the fourth authorized request with `429`

The lab limiter is per user and URI, three requests per 60 seconds.

PRODUCTION BOUNDARY

In-memory state is single-instance only. Real distributed enforcement belongs at a gateway or shared store.

Prompt 4: Complete TODO 7

Use the rate-limit prompt in `03-secure-and-limit.md`.

Require:

- Thread-safe per-caller and per-URI state
- `RateLimit-Limit`, `RateLimit-Remaining`, and `Retry-After`
- JSON error `rate_limit_exceeded` on the fourth request
- A working `reset()` for independent tests
- No sleeps, new libraries, test edits, or filter reordering

Test and Deliver

Turn design and security decisions into repeatable evidence

Seven Acceptance Tests

The suite verifies:

- Three version contracts and `v1` retirement headers
- Missing credentials return `401`
- A member calling `v3` receives `403`
- Invalid input receives `400`
- The fourth authorized request receives `429`

Run `mvn test`; no running server, network, database, or sleep is needed.

Prompt 5: Repair from Evidence

If a test fails, use the repair prompt in `04-test-and-deliver.md`.

Paste one exact failure and require the LLM to:

1. Trace only that request path
2. Compare expected status, headers, and JSON with the code
3. Propose the smallest production correction
4. Run the focused test before the full suite
5. Name the acceptance criterion restored

Inspect the Generated Contract

After all tests pass:

```
CAPSTONE_MEMBER_PASSWORD=member-pass \  
CAPSTONE_UNDERWRITER_PASSWORD=underwriter-pass \  
mvn spring-boot:run
```

Open <http://localhost:8080/swagger-ui.html> and confirm:

- All three operations appear and `v1` is deprecated
- Basic authentication is represented
- Request and response records appear as schemas

Stretch Goals

Choose one only after all seven tests pass:

1. Add a declined-decision or manual-review MockMvc case
2. Add OpenAPI responses for `400` , `401` , `403` , and `429`
3. Add the requests and assertions to a Postman collection
4. Draft soft-deprecation, hard-deprecation, and sunset communication

Stretch work stays inside the current course objectives.

Completion Checklist

- [] Audience, sensitive fields, and breaking changes identified
- [] TODOs 1-7 completed without weakening tests
- [] `v1` still works and publishes retirement headers
- [] `401`, `403`, and `429` observed on the correct requests
- [] Seven JUnit5 tests pass through `mvn test`
- [] Generated OpenAPI contract inspected
- [] One LLM-generated error caught and corrected

30-Second Share-Out

Prepare four statements:

1. One change that forced a new contract version
2. One change that could have remained non-breaking
3. The observed difference among `401` , `403` , and `429`
4. One correction you made after reviewing generated code or a failed test

Capstone Complete

Design. Evolve. Retire. Secure. Limit. Document. Test.